

Credentials & Secrets Management

1. Introduction

When we build and deploy applications using CI/CD tools like **Jenkins**, we often need **sensitive information** such as:

- Usernames & passwords
- API keys
- Tokens
- SSH keys
- Database credentials

These sensitive values are called **credentials** or **secrets**.

Important:

Never store secrets directly in:

- Source code
- Jenkinsfiles (plain text)
- GitHub repositories

That is why we need **Credentials & Secrets Management**.



2. What is Credentials & Secrets Management?

Credentials & Secrets Management is the practice of:

- Securely storing sensitive information
- Controlling who can access it
- Using it safely in applications and pipelines
- Rotating and auditing secrets

Simple Example

Instead of writing this in code:

```
DB_PASSWORD = mypassword123
```

We store the password securely and **reference it when needed**.

3. Jenkins Credentials Store

3.1 What is Jenkins Credentials Store?

The **Jenkins Credentials Store** is a **secure vault inside Jenkins** that allows you to:

- Store secrets safely
- Encrypt credentials on disk
- Use credentials in jobs and pipelines

Jenkins manages encryption automatically

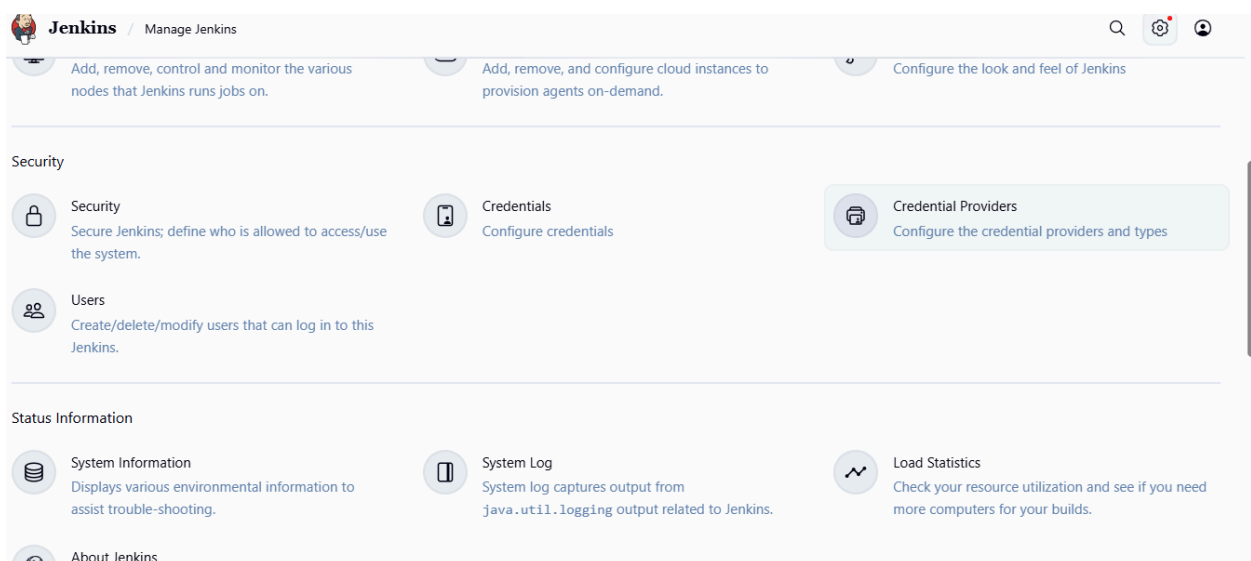
3.2 Types of Credentials in Jenkins

Jenkins supports multiple credential types:

1. **Username & Password**
 - Example: GitHub login
2. **Secret Text**
 - Example: API tokens
3. **SSH Username with Private Key**
 - Example: SSH access to servers
4. **Certificates**
 - Example: TLS/SSL certificates

3.3 Adding Credentials in Jenkins (Step-by-Step)

1. Open **Jenkins Dashboard**
2. Click **Manage Jenkins**
3. Click **Credentials**
4. Select **System** → **Global credentials (unrestricted)**
5. Click **Add Credentials**





JENKINS



Jenkins / Manage Jenkins / Credentials

Credentials

T	P	Store ↓	Domain	ID	Name
---	---	---------	--------	----	------

Stores scoped to Jenkins

P	Store ↓	Domains
	System	(global)

Icon: S M L

Jenkins / Manage Jenkins / Credentials / System / Global credentials (unrestricted)

Global credentials (unrestricted)

[+ Add Credentials](#)

Credentials that should be available irrespective of domain specification to requirements matching.

ID	Name	Kind	Description
This credential domain is empty. How about adding some credentials?			

Icon: S M L

Jenkins / Manage Jenkins / Credentials / System / Global credentials (unrestricted) / New credentials

New credentials

Kind

- Username with password
- Username with password
- GitHub App
- SSH Username with private key
- Secret file
- Secret text
- Certificate

Blank username: did you mean to use secret text credentials instead?

☐ Treat username as secret

Password

Create

Contact : +91 9966107782
info@sdlctechacademy.com



JENKINS

Fill the form:

- **Kind:** (Username/Password, Secret Text, etc.)
- **Scope:** Global
- **ID:** (example: `github-creds`)
- **Description:** Clear explanation

Click **Save**



3.4 Using Credentials in Jenkins Pipeline

Declarative Pipeline Example

```
pipeline {  
    agent any  
    stages {  
        stage('Example') {  
            steps {  
                withCredentials([  
                    usernamePassword(  
                        credentialsId: 'github-creds',  
                        usernameVariable: 'GIT_USER',  
                        passwordVariable: 'GIT_PASS'  
                    )  
                ]) {  
                    sh 'echo Using secure credentials'  
                }  
            }  
        }  
    }  
}
```



Credentials are:

- Not printed in logs
- Masked automatically

4. Why External Secrets Managers?

Jenkins credentials store is good, but in **large organizations**:

- Secrets are shared across many tools
- Secrets must be rotated automatically
- Centralized control is required

This is where **External Secrets Managers** are used.

Examples:

- HashiCorp Vault
- AWS Secrets Manager

5. HashiCorp Vault Integration

5.1 What is HashiCorp Vault?

HashiCorp Vault is a dedicated tool to:

- Securely store secrets
- Generate dynamic credentials
- Apply fine-grained access control
- Rotate secrets automatically

Vault is commonly used in **enterprise environments**.

5.2 How Jenkins Works with Vault (Concept)

Jenkins → Vault Plugin → HashiCorp Vault

Flow:

1. Jenkins authenticates to Vault
2. Vault verifies Jenkins identity
3. Jenkins requests a secret
4. Vault returns the secret securely

5.3 Jenkins + Vault Setup (High Level)

Steps:

1. Install **HashiCorp Vault Plugin** in Jenkins
2. Configure Vault URL in Jenkins
3. Configure authentication (Token / AppRole)
4. Define Vault paths
5. Use secrets in Jenkins pipeline



5.4 Sample Jenkins Pipeline with Vault

```
withVault([  
    vaultSecrets: [[  
        path: 'secret/data/myapp',  
        secretValues: [  
            [envVar: 'DB_PASS', vaultKey: 'password']  
        ]  
    ]  
]) {  
    sh 'echo Vault secret retrieved securely'  
}
```

Secrets are:

- Fetched at runtime
- Never stored permanently in Jenkins



6. AWS Secrets Manager Integration

6.1 What is AWS Secrets Manager?

AWS Secrets Manager is a managed AWS service that:

- Stores secrets securely
- Encrypts using AWS KMS
- Automatically rotates secrets
- Integrates with IAM

Commonly used in **AWS-based architectures**.

6.2 Jenkins + AWS Secrets Manager Architecture

Jenkins → AWS SDK / Plugin → AWS Secrets Manager

Authentication is done using:

- IAM Role (recommended)
- Access Key & Secret Key

6.3 Steps to Integrate Jenkins with AWS Secrets Manager

1. Create secret in **AWS Secrets Manager**
2. Attach IAM permissions to Jenkins
3. Install AWS-related Jenkins plugins
4. Configure AWS credentials in Jenkins
5. Fetch secret during pipeline execution



6.4 Example Jenkins Pipeline (AWS Secrets Manager)

```
sh '''
aws secretsmanager get-secret-value \
--secret-id my-db-secret \
--query SecretString \
--output text
'''
```

Best practice:

Use **IAM Role** instead of hardcoding AWS keys.

7. Comparison Summary

Feature	Jenkins Credentials	HashiCorp Vault	AWS Secrets Manager
Built-in	✓ Yes	✗ No	✗ No
Centralized	✗ Limited	✓ Yes	✓ Yes
Auto Rotation	✗ No	✓ Yes	✓ Yes
Best for	Small teams	Enterprise	AWS workloads



8. Best Practices

- ✓ Never hardcode secrets
- ✓ Use least privilege access
- ✓ Rotate secrets regularly
- ✓ Mask secrets in logs
- ✓ Use external secrets manager for production
- ✓ Audit access to secrets